

Trabajo Práctico Integrador

Materia: Programación II.

Integrantes: Perdomo Gabriel Maximiliano -
Sanchez Joaquin Ian.

Fecha de Entrega: 22/06/2026.

Índice

1. Introducción.....	4
2. Marco teórico	4
2.1 Java.....	4
2.2 Programación Orientada a Objetos	4
2.3 Herencia	5
2.4 Encapsulamiento	5
2.5 Interfaces.....	5
2.6 Colecciones	5
2.7 Enumeraciones	5
2.8 Manejo de excepciones	6
3. Decisiones técnicas y arquitectura.....	6
3.1 Organización del proyecto.....	6
3.2 Clase Base.....	6
3.3 Entidades principales	7
3.4 Clase Categoria	7
3.5 Clase Producto.....	7
3.6 Clase Usuario	8
3.7 Clase Pedido	8
3.8 Clase DetallePedido	8
3.9 Servicios	8
3.10 Clase Menu e interacción por consola	9
3.11 Clase Main.....	9
3.12 Baja lógica	9
3.13 Cálculo de pedidos	10
3.14 Manejo de stock.....	10
3.15 Uso de excepciones propias.....	10
4. Funcionamiento del sistema.....	10
4.1 Menú principal	11
4.2 Gestión de categorías	11
4.3 Gestión de productos	11

4.4 Gestión de usuarios.....	12
4.5 Gestión de pedidos.....	13
4.6 Validaciones y manejo de errores.....	13
5. Dificultades y soluciones.....	14
6. Conclusión.....	15
7. Bibliografía	15
8. Links del proyecto	16

1. Introducción

El presente trabajo corresponde al Trabajo Práctico Integrador de la materia Programación II de la Tecnicatura Universitaria en Programación a Distancia de la Universidad Tecnológica Nacional.

El proyecto desarrollado se denomina Food Store – Sistema de Gestión de Pedidos de Comida. Consiste en una aplicación de consola realizada en Java, orientada a la gestión de categorías, productos, usuarios y pedidos de un negocio de comidas.

El sistema permite realizar operaciones de alta, baja, modificación y consulta de datos mediante menús por consola. La información se almacena en memoria utilizando colecciones, por lo tanto, los datos existen únicamente durante la ejecución del programa.

El objetivo principal del desarrollo fue aplicar los conceptos trabajados durante la materia, como programación orientada a objetos, herencia, encapsulamiento, interfaces, colecciones, enumeraciones, manejo de excepciones y separación de responsabilidades mediante paquetes y servicios.

Durante la realización del trabajo, organizamos el proyecto en distintas capas para separar las responsabilidades del sistema. De esta manera, buscamos que el código sea claro, ordenado, modular y fácil de mantener.

2. Marco teórico

2.1 Java

Java es un lenguaje de programación orientado a objetos que permite desarrollar aplicaciones organizadas mediante clases, objetos, atributos y métodos.

En este proyecto utilizamos Java para construir un sistema de consola que permite gestionar información de un negocio de comidas. El uso de Java nos permitió aplicar conceptos fundamentales de Programación Orientada a Objetos, como encapsulamiento, constructores, herencia, sobrescritura de métodos y relaciones entre clases.

2.2 Programación Orientada a Objetos

La programación orientada a objetos permite representar elementos del mundo real mediante clases y objetos. En este sistema, las principales entidades son Categoría, Producto, Usuario, Pedido y DetallePedido.

Cada clase representa una parte del dominio del problema. Por ejemplo, la clase Producto almacena información como nombre, precio, descripción, stock, imagen, disponibilidad y categoría. La clase Usuario almacena datos como nombre, apellido, mail, celular, contraseña y rol.

Además, varias clases heredan de una clase base llamada Base, que contiene atributos comunes como id, eliminado y createdAt. Esto permite reutilizar código y mantener una estructura común entre las entidades.

2.3 Herencia

La herencia se aplicó mediante la clase abstracta Base. Esta clase contiene atributos comunes para las entidades principales del sistema, como el identificador, el estado de eliminado y la fecha de creación.

Las clases Categoría, Producto, Usuario, Pedido y DetallePedido heredan de Base. De esta manera, todas comparten una estructura general y se evita repetir atributos comunes en cada clase.

2.4 Encapsulamiento

El encapsulamiento se aplicó declarando los atributos de las clases como privados y accediendo a ellos mediante métodos getters y setters.

Esto permite proteger los datos internos de cada objeto y controlar la forma en que se modifican. Por ejemplo, los productos poseen métodos para obtener y modificar su nombre, precio, stock, descripción, imagen, disponibilidad y categoría.

2.5 Interfaces

El proyecto utiliza una interfaz llamada Calculable, que define el método calcularTotal(). Esta interfaz es implementada por la clase Pedido.

El objetivo de esta interfaz es establecer un comportamiento común para aquellas clases que deban calcular un total. En este caso, Pedido calcula su total recorriendo los detalles del pedido y sumando los subtotales.

2.6 Colecciones

Como el sistema no utiliza base de datos, la información se almacena en memoria mediante colecciones de Java. Para esto se utilizan listas, principalmente ArrayList.

Cada servicio mantiene una colección propia. Por ejemplo, CategoriaService administra una lista de categorías, ProductoService administra una lista de productos, UsuarioService administra una lista de usuarios y PedidoService administra una lista de pedidos.

2.7 Enumeraciones

El proyecto utiliza enums para representar valores fijos del sistema. Se implementaron las siguientes enumeraciones:

Rol: ADMIN y USUARIO.

Estado: PENDIENTE, CONFIRMADO, TERMINADO y CANCELADO.

FormaPago: TARJETA, TRANSFERENCIA y EFECTIVO.

El uso de enums permite evitar errores de escritura y limitar los valores posibles a opciones válidas.

2.8 Manejo de excepciones

El sistema implementa excepciones propias para controlar errores y evitar que el programa finalice inesperadamente.

Se crearon excepciones como `DatoInvalidoException`, `EntidadNoEncontradaException`, `StockInsuficienteException` y `StockInvalidoException`.

Estas excepciones permiten informar errores relacionados con datos inválidos, entidades inexistentes, falta de stock o valores incorrectos de stock y precio.

3. Decisiones técnicas y arquitectura

3.1 Organización del proyecto

Para el desarrollo del sistema decidimos organizar el proyecto en distintos paquetes, con el objetivo de separar responsabilidades y mantener el código ordenado, cumpliendo con la estructura solicitada en la consigna.

La estructura utilizada fue la siguiente:

- `entities`: contiene las clases principales del modelo, como `Base`, `Categoria`, `Producto`, `Usuario`, `Pedido` y `DetallePedido`.
- `enums`: contiene las enumeraciones `Estado`, `FormaPago` y `Rol`.
- `exception`: contiene las excepciones propias del sistema.
- `interfaces`: contiene la interfaz `Calculable`.
- `service`: contiene las clases encargadas de la lógica del sistema, como `CategoriaService`, `ProductoService`, `UsuarioService` y `PedidoService`.
- `view`: contiene la clase `Menu`, encargada de mostrar las opciones por consola y recibir los datos ingresados por el usuario.
- `TrabajoPracticoIntegrador`: contiene la clase `Main`, que inicia la ejecución del programa.

Esta organización nos permitió separar el modelo de datos, la lógica de negocio, las excepciones, las interfaces y la interacción con el usuario. De esta manera, el código quedó más claro, modular y fácil de mantener.

3.2 Clase Base

Implementamos una clase abstracta llamada `Base` para representar los atributos comunes de las entidades del sistema. Esta clase contiene los campos `id`, `eliminado` y `createdAt`.

El campo id se utiliza para identificar cada registro. El campo eliminado permite aplicar baja lógica, evitando borrar físicamente los objetos de las colecciones. El campo createdAt guarda la fecha y hora de creación del objeto.

Todas las entidades principales del sistema heredan de esta clase, lo que permite compartir una estructura común y evitar la repetición de código.

3.3 Entidades principales

Definimos distintas entidades para representar los elementos principales del sistema.

La clase Categoría representa las categorías de productos. Contiene nombre y descripción.

La clase Producto representa los productos del negocio. Contiene nombre, precio, descripción, stock, imagen, disponibilidad y una relación con Categoría.

La clase Usuario representa a los usuarios del sistema. Contiene nombre, apellido, mail, celular, contraseña y rol.

La clase Pedido representa una compra o pedido realizado por un usuario. Contiene fecha, estado, total, forma de pago, usuario asociado y una lista de detalles.

La clase DetallePedido representa cada producto incluido dentro de un pedido. Contiene cantidad, subtotal y producto asociado.

Estas clases reflejan el modelo planteado en la consigna y permiten representar el funcionamiento básico de un sistema de pedidos de comida.

3.4 Clase Categoría

La clase Categoría hereda de Base y representa una categoría dentro del catálogo de productos. Sus atributos principales son nombre y descripción.

Esta clase permite agrupar productos según su tipo. Por ejemplo, se podrían crear categorías como hamburguesas, pizzas, bebidas o postres.

Además, la clase cuenta con métodos getters y setters para acceder a sus atributos, un método toString() para mostrar la información por consola y métodos equals() y hashCode() para comparar categorías mediante su id.

3.5 Clase Producto

La clase Producto hereda de Base y representa los productos disponibles dentro del negocio de comidas.

Sus atributos principales son nombre, precio, descripción, stock, imagen, disponible y categoría. La relación con Categoría permite asociar cada producto a una categoría determinada.

También se implementaron métodos getters y setters, un método toString() para mostrar los datos principales del producto y métodos equals() y hashCode() para comparar productos por id.

3.6 Clase Usuario

La clase Usuario hereda de Base y representa a los usuarios del sistema. Sus atributos son nombre, apellido, mail, celular, contraseña y rol.

El atributo rol utiliza el enum Rol, que permite diferenciar entre usuarios ADMIN y USUARIO.

La clase también cuenta con métodos getters y setters, un método toString() para mostrar la información del usuario y métodos equals() y hashCode() para comparar usuarios por id.

3.7 Clase Pedido

La clase Pedido hereda de Base e implementa la interfaz Calculable. Representa un pedido realizado por un usuario.

Sus atributos son fecha, estado, total, formaPago, usuario y una lista de detalles. El estado del pedido utiliza el enum Estado, mientras que la forma de pago utiliza el enum FormaPago.

La clase Pedido contiene métodos para agregar detalles, buscar un detalle por producto, eliminar un detalle por producto y calcular el total del pedido.

El método calcularTotal() recorre la lista de detalles y suma los subtotales para obtener el total final.

3.8 Clase DetallePedido

La clase DetallePedido hereda de Base y representa cada línea o ítem dentro de un pedido.

Sus atributos son cantidad, subtotal y producto. Cada detalle está asociado a un producto específico y guarda la cantidad solicitada y el subtotal correspondiente.

El subtotal se calcula multiplicando la cantidad por el precio del producto.

3.9 Servicios

Para separar la lógica de negocio de la interacción por consola, implementamos clases de servicio. Cada servicio administra una colección en memoria y ofrece métodos para crear, listar, buscar, editar y eliminar datos.

CategoriaService permite crear, listar, buscar, editar y eliminar categorías. Además, valida que el nombre no esté vacío y que no existan categorías repetidas.

ProductoService permite crear, listar, buscar, editar y eliminar productos. También valida que el nombre no esté vacío, que el precio no sea negativo y que el stock no sea negativo.

UsuarioService permite crear, listar, buscar, editar y eliminar usuarios. Además, valida que el nombre, apellido y mail no estén vacíos, y que el mail sea único.

PedidoService permite crear pedidos, listar pedidos, buscar pedidos por id, agregar detalles, actualizar estado, actualizar forma de pago y eliminar pedidos. También valida que la cantidad sea mayor a cero y que exista stock suficiente antes de agregar un producto al pedido.

3.10 Clase Menu e interacción por consola

La interacción con el usuario se realiza mediante la clase Menu, ubicada en el paquete view. Esta clase utiliza Scanner para leer los datos ingresados por consola y muestra un menú principal con las opciones principales del sistema.

El menú principal permite acceder a:

1. Categorías
2. Productos
3. Usuarios
4. Pedidos
5. Salir

Cada opción abre un submenú específico. Por ejemplo, el menú de categorías permite listar, crear, editar y eliminar categorías. Lo mismo ocurre con productos, usuarios y pedidos.

La clase Menu se encarga de solicitar los datos al usuario, llamar a los servicios correspondientes y mostrar mensajes de éxito o error. Para evitar que el programa se cierre inesperadamente, protegimos las operaciones principales con bloques try-catch.

Toda la interacción ocurre por consola mediante menús, y cada sección permite realizar operaciones CRUD sobre las colecciones en memoria.

3.11 Clase Main

La clase Main es el punto de entrada del programa. Su responsabilidad es crear un objeto de tipo Menu y ejecutar el método iniciar().

De esta manera, Main no contiene reglas de negocio complejas, sino que solamente inicia la aplicación. Esto respeta la separación de responsabilidades, ya que la interacción con el usuario se concentra en Menu y la lógica del sistema se ubica en los servicios.

3.12 Baja lógica

Una decisión importante fue utilizar baja lógica en lugar de eliminar físicamente los objetos de las colecciones.

Cuando se elimina una categoría, producto, usuario o pedido, el sistema marca el atributo eliminado como true. Luego, en los listados, solamente se muestran los elementos que no están eliminados.

Esto permite mantener el historial de datos durante la ejecución y evita inconsistencias entre objetos relacionados.

3.13 Cálculo de pedidos

La clase Pedido implementa la interfaz Calculable. Cada vez que se agrega un detalle al pedido, se calcula el subtotal del detalle y luego se actualiza el total del pedido.

El total se obtiene recorriendo la lista de detalles y sumando los subtotales. De esta manera, el pedido mantiene actualizado el importe total de la compra.

3.14 Manejo de stock

Al agregar un producto a un pedido, el sistema valida que la cantidad solicitada sea mayor a cero y que el producto tenga stock suficiente.

Si la validación es correcta, se agrega el detalle al pedido y se descuenta la cantidad correspondiente del stock del producto.

Si no hay stock suficiente, se lanza una excepción y se informa el error.

3.15 Uso de excepciones propias

Implementamos excepciones propias para controlar situaciones específicas del sistema.

`DatoInvalidoException` se utiliza cuando se ingresan datos vacíos o incorrectos.

`EntidadNoEncontradaException` se utiliza cuando se intenta buscar una categoría, producto, usuario o pedido que no existe o se encuentra eliminado.

`StockInsuficienteException` se utiliza cuando se intenta agregar a un pedido una cantidad mayor al stock disponible.

`StockInvalidoException` se utiliza cuando se intenta crear o editar un producto con precio o stock negativo.

El uso de estas excepciones permite que el sistema muestre mensajes claros y continúe funcionando sin cerrarse inesperadamente.

4. Funcionamiento del sistema

El sistema inicia desde la clase Main. En esta clase se crea un objeto de tipo Menu y se ejecuta el método `iniciar()`, que muestra el menú principal de la aplicación.

Desde ese menú, el usuario puede ingresar a las distintas secciones del sistema: categorías, productos, usuarios y pedidos. Cada sección posee su propio submenú con opciones para listar, crear, editar y eliminar registros.

El sistema trabaja completamente en consola y almacena los datos en memoria durante la ejecución del programa mediante colecciones.

4.1 Menú principal

Al ejecutar el programa se muestra el menú principal de Food Store. Desde allí se puede acceder a las distintas funcionalidades del sistema.

```
=== FOOD STORE ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
```

4.2 Gestión de categorías

En la gestión de categorías, el sistema permite listar las categorías cargadas, crear nuevas categorías, editar una categoría existente y eliminarla de manera lógica.

Al crear una categoría, se solicita el nombre y la descripción. El sistema valida que el nombre no esté vacío y que no exista otra categoría con el mismo nombre.

```
=== FOOD STORE ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Selección: 1

=== CATEGORÍAS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Selección: 2
Nombre: Hamburguesas
Descripción: Hamburguesas y empanadas
Categoría creada con id 1

=== CATEGORÍAS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Selección: 3
Categoría(id=1, nombre='Hamburguesas', descripcion='Hamburguesas y empanadas')
Id: 1
Nuevo nombre: Comidas rápidas
Nueva descripción: Hamburguesas y Empanadas
Categoría modificada correctamente
```

4.3 Gestión de productos

En la gestión de productos, el sistema permite crear productos asociados a una categoría. Para cada producto se solicita nombre, precio, descripción, stock, imagen, disponibilidad y categoría.

El sistema valida que el nombre no esté vacío, que el precio no sea negativo y que el stock no sea negativo. Además, antes de crear o editar un producto, se listan las categorías para seleccionar una categoría existente.

```

=== FOOD STORE ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 2

=== PRODUCTOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Categoría(id=1, nombre='Comidas rápidas', descripción='Hamburguesas y Empanadas')
Id categoría: 1
Nombre: Hamburguesa cuádruple cheddar
Precio: 500
Descripción: Hamburguesa doble carne con cuádruple cheddar
Stock: 30
Imagen: 1
Disponible (true/false): true
Producto creado con id 1

=== PRODUCTOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
Producto{id=1, nombre='Hamburguesa cuádruple cheddar', precio=500.0, stock=30, categoría=Comidas rápidas}

```

4.4 Gestión de usuarios

En la gestión de usuarios, el sistema permite crear usuarios con nombre, apellido, mail, celular, contraseña y rol.

El sistema valida que el nombre, apellido y mail no estén vacíos. También valida que el mail sea único, evitando cargar dos usuarios con el mismo correo electrónico.

```

=== FOOD STORE ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 3

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Joaco
Apellido: Sanchez
Mail: sánchezjoaco@gmail.com
Celular: 113231
Contraseña: 1208
1. ADMIN
2. USUARIO
Rol: 1
Usuario creado con id 1

```

4.5 Gestión de pedidos

En la gestión de pedidos, el sistema permite crear pedidos asociados a un usuario existente. Luego, se pueden agregar productos al pedido indicando la cantidad deseada.

El sistema valida que la cantidad sea mayor a cero y que el producto tenga stock suficiente. Al agregar un producto, se calcula el subtotal del detalle y se actualiza el total del pedido mediante el método `calcularTotal()`.

También se permite actualizar el estado del pedido y la forma de pago, utilizando las enumeraciones `Estado` y `FormaPago`.

```
=== PEDIDOS ===
1. Listar
2. Crear
3. Actualizar estado
4. Actualizar forma de pago
5. Eliminar
0. Volver
Seleccione: 2
Usuario{id=1, nombre='Joaco', apellido='Sanchez', mail='sanchezjoaco@gmail.com', rol=ADMIN}
Id usuario: 1
Forma de pago
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
2
Producto{id=1, nombre='Hamburguesa cuadruple cheddar', precio=500.0, stock=30, categoria=Comidas rapidas}
Id producto: 1
Cantidad: 2
Agregar otro producto? (S/N): N
Pedido creado correctamente
Total: $1000.0
```

4.6 Validaciones y manejo de errores

El sistema utiliza bloques try-catch en las operaciones del menú para capturar errores y mostrar mensajes al usuario sin finalizar la ejecución del programa.

Por ejemplo, si se intenta buscar una entidad inexistente, ingresar datos inválidos o cargar una cantidad mayor al stock disponible, el sistema informa el error correspondiente y permite continuar utilizando el menú.

```
=== PEDIDOS ===
1. Listar
2. Crear
3. Actualizar estado
4. Actualizar forma de pago
5. Eliminar
0. Volver
Seleccione: 2
Usuario{id=1, nombre='Joaco', apellido='Sanchez', mail='sanchezjoaco@gmail.com', rol=ADMIN}
Id usuario: 2
Usuario no encontrado
```

```

=== FOOD STORE ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 3

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Gabi
Apellido: Perdomo
Mail: sanchezjoaco@gmail.com
Celular: 123123
Contraseña: 9999
1. ADMIN
2. USUARIO
Rol: 2
El mail ya existe

```

```

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 4
Usuario{id=1, nombre='Joaco', apellido='Sanchez', mail='sanchezjoaco@gmail.com', rol=ADMIN}
Id usuario: 0
Confirma eliminación? (S/N): s
Usuario no encontrado

```

```

=== PEDIDOS ===
1. Listar
2. Crear
3. Actualizar estado
4. Actualizar forma de pago
5. Eliminar
0. Volver
Seleccione: 2
Usuario{id=1, nombre='Joaco', apellido='Sanchez', mail='sanchezjoaco@gmail.com', rol=ADMIN}
Id usuario: 1
Forma de pago
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
3
Producto{id=1, nombre='Hamburguesa cuádruple cheddar', precio=500.0, stock=28, categoria=Comidas rápidas}
Id producto: 1
Cantidad: 30
Stock insuficiente

```

5. Dificultades y soluciones

Durante el desarrollo del sistema surgieron algunas dificultades técnicas.

Una de las principales dificultades fue organizar correctamente las clases y separar las responsabilidades. Para resolverlo, dividimos el proyecto en paquetes, separando entidades, servicios, enumeraciones, excepciones, interfaces y vista.

Otra dificultad fue manejar las validaciones sin repetir demasiada lógica en el menú. Para solucionarlo, ubicamos las validaciones principales dentro de los servicios. De esta forma, el menú se encarga de pedir datos al usuario y los servicios se encargan de controlar las reglas de negocio.

También fue necesario controlar errores como datos vacíos, stock negativo, precio negativo, entidades inexistentes o stock insuficiente. Para esto implementamos excepciones propias, que permiten mostrar mensajes claros al usuario sin detener la ejecución del programa.

Otra dificultad fue mantener actualizado el total del pedido. Para resolverlo, implementamos la interfaz Calculable en la clase Pedido y utilizamos el método `calcularTotal()` cada vez que se agregan o eliminan detalles.

Además, trabajamos con baja lógica para no eliminar físicamente los objetos de las colecciones. Esto permitió mantener la información durante la ejecución y evitar problemas con entidades relacionadas, como usuarios asociados a pedidos o productos incluidos en detalles de pedidos.

También fue importante separar correctamente la interacción de consola de la lógica del sistema. Para resolverlo, dejamos la clase Menu encargada de pedir datos y mostrar opciones, mientras que las clases Service concentran las validaciones y operaciones principales.

6. Conclusión

El desarrollo del sistema Food Store nos permitió aplicar los principales contenidos de la materia Programación II en un proyecto integrador.

A través del sistema trabajamos con clases, objetos, herencia, encapsulamiento, interfaces, enumeraciones, colecciones, excepciones propias y separación por capas.

El resultado final es una aplicación de consola capaz de gestionar categorías, productos, usuarios y pedidos, respetando reglas básicas de negocio y manteniendo una estructura de código ordenada.

El proyecto nos permitió comprender la importancia de organizar correctamente el código, validar los datos ingresados por el usuario y separar la lógica de negocio de la interacción por consola.

Además, el trabajo en equipo nos permitió dividir tareas, revisar el código entre ambos integrantes y mejorar la estructura general del sistema.

7. Bibliografía

Oracle. Documentación oficial de Java.
<https://docs.oracle.com/en/java/>

Oracle. Documentación de ArrayList.

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html>

Oracle. Documentación de LocalDate y LocalDateTime.

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/package-summary.html>

Oracle. Documentación sobre excepciones en Java.

<https://docs.oracle.com/javase/tutorial/essential/exceptions/>

Oracle. Documentación de Scanner.

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Scanner.html>

Material de cátedra de Programación II – UTN-TUPaD.

8. Links del proyecto

Repositorio GitHub: <https://github.com/JoakitoLDZ/TrabajoIntegradorProgramacion2>

<https://github.com/gabiperdo7/TrabajoIntegradorProgramacion2>

Video demostrativo: https://drive.google.com/drive/folders/11akFu9zRT_yi-gNo2090bILW05Gpiy-W?usp=sharing